一、Math对象

语法： `Math.random()`

作用： 随机生成0-1（包含0但不包含1）的随机数

示例：

```
console.log(Math.random())
```

向下取整： `Math.floor()`

示例：

```
console.log(Math.floor(Math.random()))
```

1.1 练习

案例： 年会抽奖

需求： 从数组中随机抽取一等奖，显示对应的标签里。

数组名单：

```
const personArr = ['周杰伦', '刘德华', 'Viking' '周星驰', '彭于晏']
```

年会抽奖：

一等奖：彭于晏

二、定时器

方法	说明
`setInterval(fn, interval)`	每隔 `interval` 毫秒重复执行 `fn`
`setTimeout(fn, delay)`	延迟 `delay` 毫秒后执行 `fn` （只执行一次）
`clearTimeout(id)` / `clearInterval(id)`	取消定时器

2.1 间歇定时器

语法：

```
setInterval(函数, 间隔时间)
```

作用：

- 每隔一段时间调用这个函数
- 间隔时间单位是毫秒

示例：

```
let timer = setInterval(function(){
    console.log('hello');
}, 2000)
```

关闭定时器：

```
let 变量名 = setInterval(函数, 间隔时间)
clearInterval(变量名)
```

一般不会刚创建就停止，而是满足一定条件在停止

2.2 延迟定时器

语法：

```
setTimeout(函数, 延迟时间)
```

作用：

- 延迟一段时间后调用这个函数
- 间隔时间单位是毫秒

示例：

```
setTimeout(function(){
    console.log('hello');
}, 2000)
```

关闭定时器：

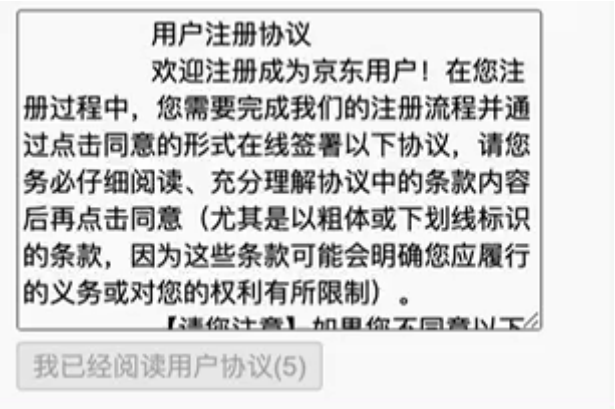
```
let timer = setTimeout(function(){
    console.log('hello');
}, 2000)

// 关闭定时器
clearTimeout(timer)
```

一般不会刚创建就停止，而是满足一定条件在停止

2.3 练习

案例： 用户阅读协议倒计时效果

示例：




```
let btn = document.querySelector('button')
        let i = 5
        let sil = setInterval(function(){
            if (i === 1){
                btn.disabled = false
                btn.innerHTML = '我已经阅读用户协议'
                clearInterval(sil)
            }
            else{
                i--
                btn.innerHTML = `我已经阅读用户协议(${i})`
            }
        }, 1000)
```

三、操作元素属性

3.1 一般属性

介绍：

- 通过 JS 设置/修改标签元素属性，比如通过 src更换 图片

- 最常见的属性比如： href、title、src 等

语法：

```
对象.属性 = 值
```

示例：

```
let my_img = document.querySelector('img')
my_img.src = './logo.png'
my_img.title = 'logo图标'
```

3.2 样式属性

语法：

```
对象.style.样式属性 = 值
```

示例：

```
const box = document.querySelector('.box')
// 修改元素样式
box.style.width = '200px'
box.style.marginTop = '15px'
box.style.backgroundColor = 'pink'
```

> **注意：**
>
> 1. 修改样式通过style属性引出
> 2. 如果属性有-连接符，需要转换为小驼峰命名法
> 3. 赋值的时候，需要的时候不要忘记加css单位

3.3 class属性

介绍：

如果修改的样式比较多，直接通过style属性修改比较繁琐，我们可以通过借助于css类名的形式

语法:

```
// active 是一个css类名
元素.className = 'active'
```

注意:

1. 由于class是关键字, 所以使用className去代替
2. className是使用新值换旧值, 如果需要添加一个类,需要保留之前的类名

3.4 classList属性

介绍:

为了解决className 容易覆盖以前的类名，我们可以通过classList方式追加和删除类名

语法:

```
// 追加一个类
元素.classList.add('类名')
// 删除一个类
元素.classList.remove('类名')
// 切换一个类
元素.classList.toggle('类名')
```

四、添加与删除DOM

4.1 制作DOM

通过 `document.createElement("标签名")` 可以创建一个dom对象。

```
let div = document.createElement("div")
```

4.2 插入DOM

创建的新元素需要插入到 DOM 树中才能显示在页面上。可以使用以下方法将新元素插入到指定位置:

4.2.1 追加到父元素末尾

使用 `appendChild` 方法将新元素添加到父元素的子元素列表末尾。

```
// 获取 body 元素
const body = document.body;

// 将新元素添加到 body 的末尾
body.appendChild(newParagraph);
```

4.2.2 指定位置插入

- `insertBefore` 方法允许你将一个新元素插入到父元素的子元素列表中的某个特定位置。
- 它的作用是将新元素插入到参考元素之前。

语法

```
parentNode.insertBefore(newNode, referenceNode);
```

- `parentNode`：要插入新元素的父元素。
- `newNode`：要插入的新元素。
- `referenceNode`：参考元素，新元素将插入到这个元素之前。如果为 `null`，则新元素会被追加到父元素的末尾（类似于 `appendChild`）。

```
<ul id="list">
    <li id="item1">第一项</li>
    <li id="item2">第二项</li>
</ul>

<script>
    // 获取父元素和参考元素
    const list = document.querySelector('#list');
    const item2 = document.querySelector('#item2');

    // 创建新元素
    const newItem = document.createElement('li');
    newItem.textContent = '插入的新项';

    // 将新元素插入到 item2 之前
    list.insertBefore(newItem, item2);
</script>
```

4.3 总结

- `appendChild`：将新元素追加到父元素的末尾。
- `insertBefore`：将新元素插入到指定元素之前。

4.4 删除DOM

4.5 remove

直接调用元素的 `remove` 方法来删除自身。

```
// 获取要删除的元素
const paragraph = document.querySelector('p');

// 直接删除元素
paragraph.remove();
```

4.6 获取父元素

`parentNode`：获取到该元素的父元素对象

```
// 获取要删除的元素
const paragraph = document.querySelector('p');

// 获取到父元素，并将整个父元素删除
paragraph.parentNode.remove(paragraph);
```

五、本地存储

5.1 什么是本地存储?

5.1.1 定义

- **本地存储** 是 HTML5 引入的一个 Web API，用于在浏览器中存储数据。
- 数据以字符串形式存储，且没有过期时间限制（除非手动清除）。
- 存储容量通常为 **5MB**（具体大小因浏览器而异）。

5.1.2 特点

1. **持久性**：
   - 数据会一直保留在用户的设备上，即使关闭浏览器或重启电脑也不会丢失。
2. **作用域**：
   - 数据在同一域名下共享，不同域名之间的存储是隔离的。

3. **简单易用**：
   - 提供了简单的 API（`setItem`、`getItem` 等），便于操作。

5.2 本地存储的基本操作

本地存储的核心 API 是 `localStorage` 对象，它提供了以下常用方法：

5.2.1 存储数据

```
localStorage.setItem('key', 'value');
```

- 将数据以键值对的形式存储到本地存储中。
- 如果键已存在，则会覆盖原来的值。

示例

```
localStorage.setItem('username', 'Alice');
```

5.2.2 获取数据

```
const value = localStorage.getItem('key');
```

- 根据键名获取存储的数据。
- 如果键不存在，则返回 `null`。

示例

```
const username = localStorage.getItem('username');
console.log(username); // 输出：Alice
```

5.2.3 删除单个数据

```
localStorage.removeItem('key');
```

- 根据键名删除对应的存储数据。

示例

```
localStorage.removeItem('username');
```

5.2.4 清空所有数据

```
localStorage.clear();
```

- 清空当前域名下的所有本地存储数据。

5.2.5 遍历存储的数据

可以通过 `localStorage.length` 和 `localStorage.key(index)` 遍历所有的键值对。

示例

```
for (let i = 0; i < localStorage.length; i++) {
    const key = localStorage.key(i);
    const value = localStorage.getItem(key);
    console.log(`${key}: ${value}`);
}
```

5.3 注意事项

5.3.1 数据类型

- `localStorage` 只能存储字符串。
- 如果需要存储对象或数组等复杂数据类型，必须先将其转换为 JSON 字符串。

5.3.1.1 示例

```
// 存储对象
const user = { name: 'Alice', age: 25 };
localStorage.setItem('user', JSON.stringify(user));

// 获取对象
const storedUser =
JSON.parse(localStorage.getItem('user'));
console.log(storedUser); // 输出：{ name: 'Alice', age: 25 }
```

5.3.2 安全性

- **不要存储敏感信息**：
  - `localStorage` 中的数据未加密，容易被恶意脚本（如 XSS 攻击）窃取。
  - 不要存储密码、令牌等敏感信息。
- **同源策略**：

- 数据只能在同一域名下访问，不同域名之间无法共享。

5.4 本地存储与会话存储的区别

除了 `localStorage`，HTML5 还提供了另一种存储方式：**Session Storage**。两者的主要区别如下：

特性	`localStorage`	`sessionStorage`
生命周期	持久化存储，直到手动清除	仅在当前会话期间有效，关闭标签页后清除
作用域	同一域名下的所有窗口共享	仅限于当前浏览器窗口
存储容量	通常为 5MB	通常为 5MB
适用场景	需要长期保存的数据	临时数据，例如表单状态

5.5 实战案例

5.5.1 案例 1：保存用户偏好设置

假设我们有一个主题切换功能，用户可以选择浅色模式或深色模式，并希望在页面刷新后仍然保留选择。

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>主题切换</title>
    <style>
        body.light {
            background-color: #fff;
            color: #000;
        }
```

```
        body.dark {
            background-color: #333;
            color: #fff;
        }
    </style>
</head>
<body class="light">
    <h1>主题切换示例</h1>
    <button id="toggleTheme">切换主题</button>

    <script>
        // 加载时读取存储的主题
        const savedTheme = localStorage.getItem('theme');
        if (savedTheme) {
            document.body.className = savedTheme;
        }

        // 切换主题并保存到 localStorage

 document.getElementById('toggleTheme').addEventListener('cl
ick', function() {
            if (document.body.className === 'light') {
                document.body.className = 'dark';
                localStorage.setItem('theme', 'dark');
            } else {
                document.body.className = 'light';
                localStorage.setItem('theme', 'light');
            }
        });
    </script>
</body>
</html>
```

5.5.2 案例 2：购物车功能

使用 `localStorage` 保存用户的购物车数据。

```
// 添加商品到购物车
function addToCart(productId, productName) {
    let cart = JSON.parse(localStorage.getItem('cart')) ||
[];
    cart.push({ id: productId, name: productName });
    localStorage.setItem('cart', JSON.stringify(cart));
}

// 获取购物车数据
function getCart() {
    return JSON.parse(localStorage.getItem('cart')) || [];
}

// 示例
addToCart(1, 'iPhone');
addToCart(2, 'MacBook');
console.log(getCart()); // 输出：[{ id: 1, name: 'iPhone' },
{ id: 2, name: 'MacBook' }]
```

5.6 总结

5.6.1 关键点

- `localStorage` 是一种持久化存储机制，适合保存不需要频繁更改的数据。
- 数据以字符串形式存储，复杂数据类型需要通过 `JSON.stringify` 和 `JSON.parse` 转换。
- 避免存储敏感信息，确保数据安全。

5.6.2 适用场景

- 用户偏好设置（如主题、语言等）。
- 临时数据缓存（如表单状态、购物车内容等）。
- 离线应用中的数据存储。